Recruitment Management System

State Machine Diagrams

Ten diagrams covering every stateful entity

Transition tables · Guards · Invalid-transition rules

Implementation patterns and interview preparation

UML state machine notation · Rendered with Graphviz

1. What a State Diagram Adds

You now have three views of the same system. Each answers a different question, and being able to say which is which is worth doing early in any viva.

Diagram	Question answered	Maps onto
ER	What is stored?	Tables, columns, foreign keys
DFD	What moves, and what transforms it?	Processes, services, API endpoints
State	What can happen next, and what may not?	`status` enum columns and the transitions your service layer permits

A state diagram is the direct visual form of your `status` columns. Every state below is a literal value from an enum in your schema — nothing is invented for the diagram. That correspondence is the point: the diagram is a specification your code must satisfy, not decoration.

1.1 Notation

● **Initial pseudostate** — where the object begins. Not a state; the object never rests here.

◎ **Final state** — the object's lifecycle ends. No further transitions.

State (neutral) **Success** **Terminal / failure** **Awaiting action** **Funds held**

Transition label format: `event [guard] / action` — the event that triggers it, the condition that must hold, and the side effect performed. A transition with no event is automatic once the guard is satisfied.

> **Why this matters for implementation.** A state diagram tells you which transitions to *reject*. Most bugs in workflow systems are not wrong data — they are transitions that should have been impossible. If a milestone can move from `PENDING` straight to `PAID` because nothing checks, you have paid for work that was never delivered. The diagram is the list of things your service layer must refuse.

2. Job Posting

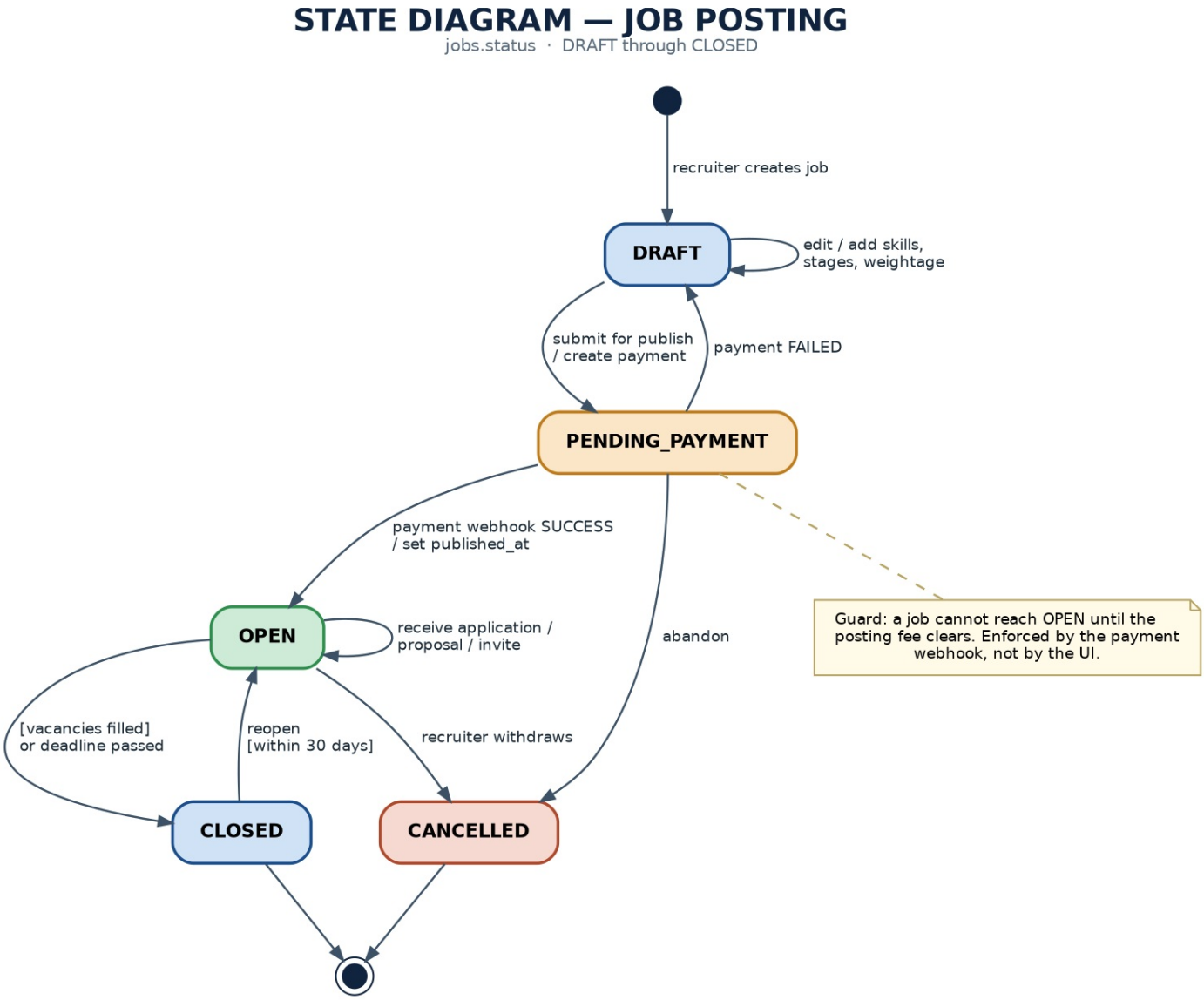


Figure 1 — `jobs.status`

State	Meaning	Permitted next
`DRAFT`	Being composed; not visible to anyone but its author	`PENDING_PAYMENT`
`PENDING_PAYMENT`	Submitted but the posting fee has not cleared	`OPEN`, `DRAFT`, `CANCELLED`
`OPEN`	Live and accepting applications or proposals	`CLOSED`, `CANCELLED`
`CLOSED`	Filled or past deadline; no longer accepting	`OPEN` (reopen)
`CANCELLED`	Abandoned by the recruiter	— terminal

The design point worth stating: `PENDING_PAYMENT` exists so that visibility is gated on money clearing rather than on a UI flag. The transition to `OPEN` is driven by the payment webhook, so a recruiter cannot publish by manipulating the client.

3. Application — the Full-Time Pipeline

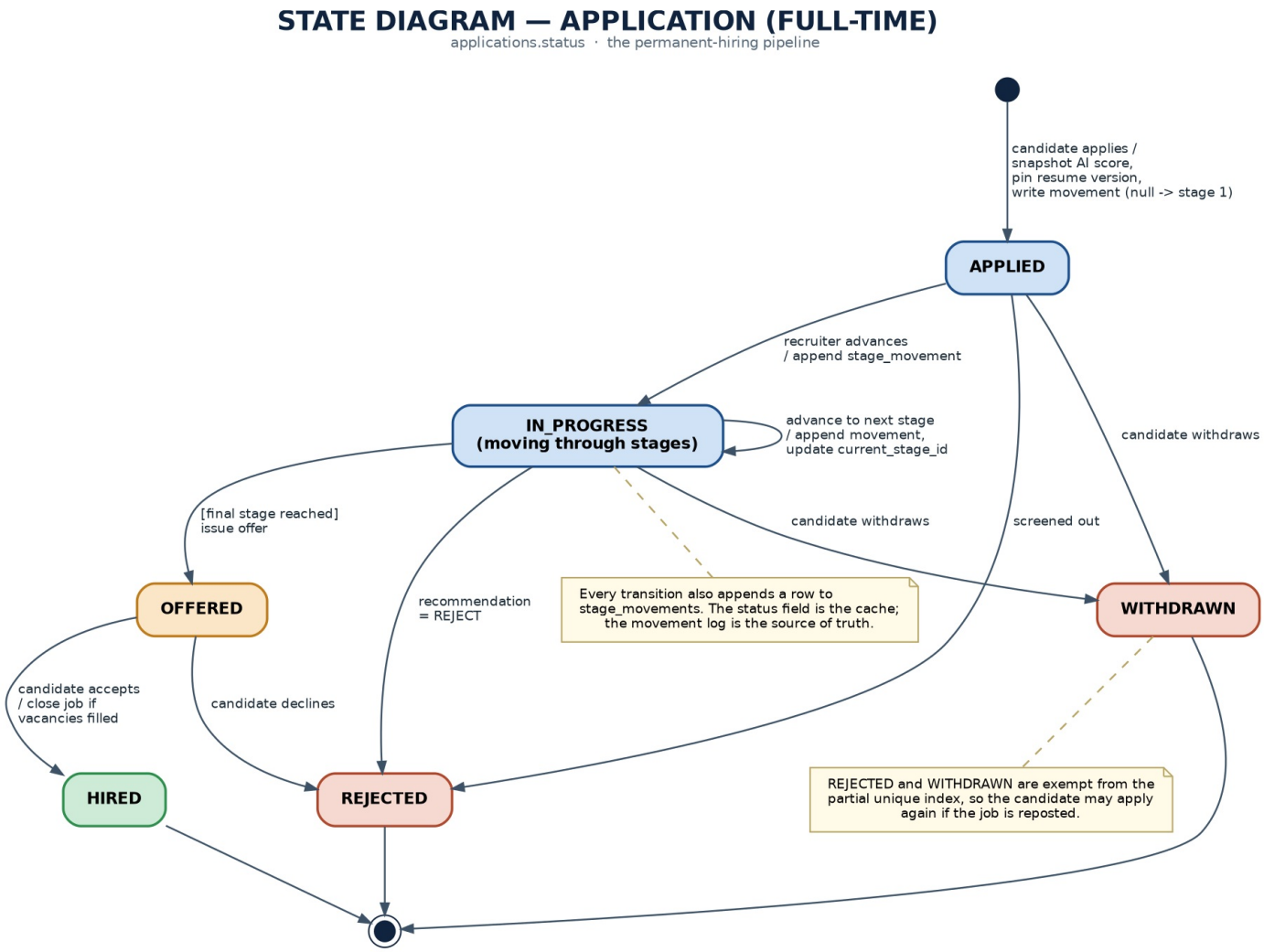


Figure 2 — `applications.status`

State	Meaning	Permitted next
`APPLIED`	Submitted, sitting at the first stage	`IN_PROGRESS`, `REJECTED`, `WITHDRAWN`
`IN_PROGRESS`	Moving through recruiter-defined stages	`OFFERED`, `REJECTED`, `WITHDRAWN`
`OFFERED`	Offer extended, awaiting the candidate	`HIRED`, `REJECTED`
`HIRED`	Offer accepted	— terminal
`REJECTED` / `WITHDRAWN`	Ended by recruiter or candidate	— terminal

Two things to point out about this diagram.

The self-loop on `IN_PROGRESS` hides a sub-machine. Stage movement is its own state machine, but the states are *data* — rows in `hiring_stages` configured per job — not enum values. You cannot draw a fixed diagram for a pipeline the recruiter defines at runtime, which is exactly why `stage_movements` exists as a table rather than as more enum values.

The terminal states are not equally terminal. `REJECTED` and `WITHDRAWN` are excluded from the partial unique index, so the candidate can apply again if the role is reposted. `HIRED` is genuinely final. Same-looking box, different consequence.

4. Invitation and Offer

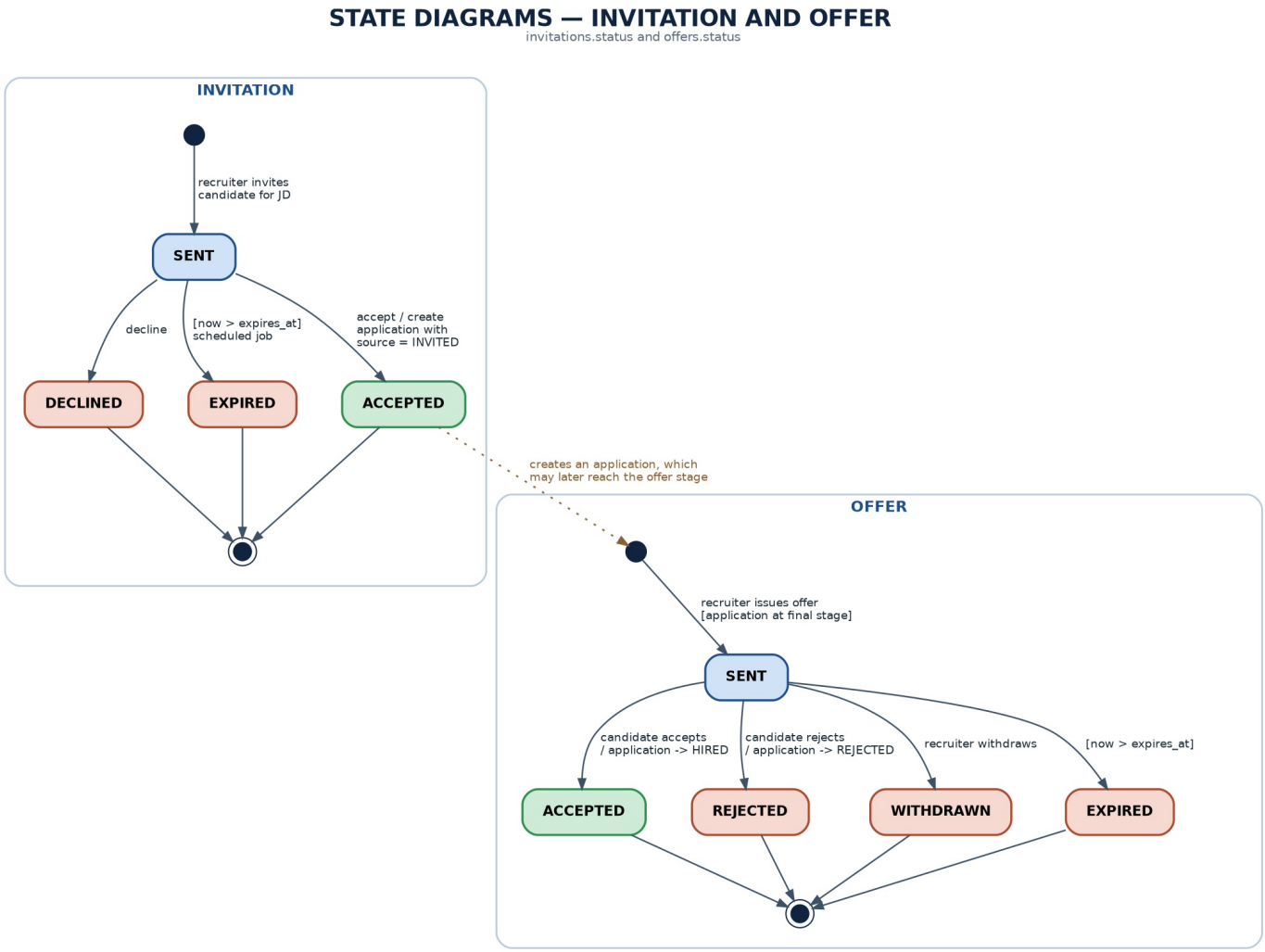


Figure 3 — `invitations.status` and `offers.status`

Both are short-lived request-response lifecycles with a timeout, and both are worth having separate from the application. An invitation is something the candidate has *not yet consented to*; collapsing it into an application would place people in pipelines they never joined. An offer needs its own `expires_at` because offers lapse commercially, and modelling that in the database rather than in a cron job's assumptions keeps the state honest.

The `EXPIRED` transitions are time-driven, not user-driven. A scheduled job sweeps rows past `expires_at`. Worth naming, because a transition with no actor is easy to forget to implement, and its absence shows up as invitations that stay open forever.

5. Proposal to Contract — the Critical Boundary

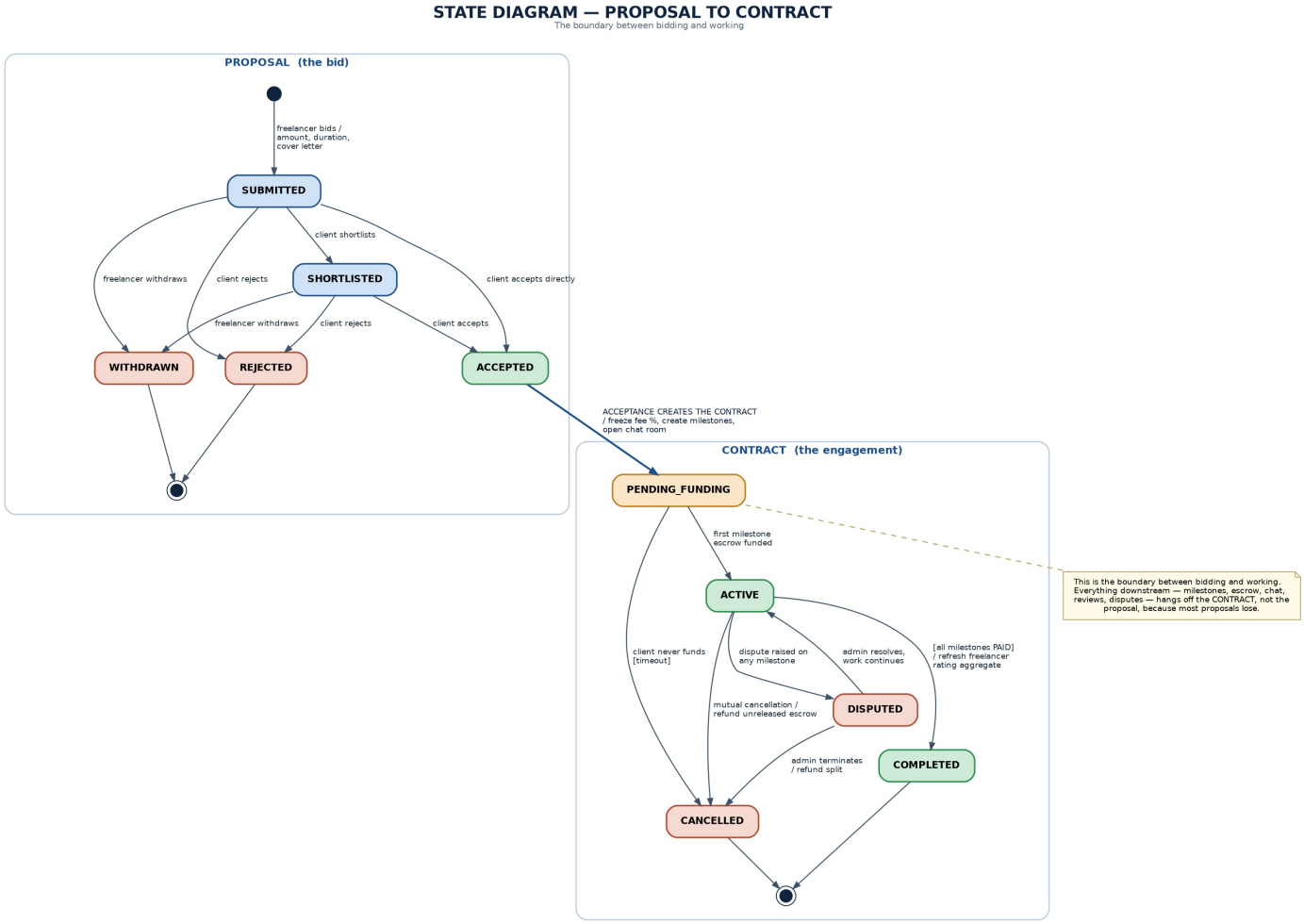


Figure 4 — `proposals.status` and `contracts.status`

The bold arrow is the single most important transition in the system. A proposal reaching `ACCEPTED` does not merely change a status — it **creates a new object with its own lifecycle**. In that one transaction the contract is created, the platform fee percentage is frozen at the current rate, the milestone rows are generated, and the chat room is opened with both parties as participants.

> **Why this is the right place to draw the boundary.** A proposal is a bid, and most bids lose. If milestones, escrow, chat and reviews hung off the proposal, they would be attached to an object that is usually rejected — and there would be nowhere to store the negotiated amount, which can differ from the bid, or the frozen fee rate, which must not change if the platform later alters its pricing. Two objects with two lifecycles is the correct model, and the acceptance transition is the seam between them.

Note also that `DISPUTED` on the contract is *not* terminal — a contract can return to `ACTIVE` after an admin resolves the dispute, because the remaining milestones still need doing. Modelling dispute as a detour rather than an ending is a small but meaningful correctness point.

6. Milestone — the Core Machine

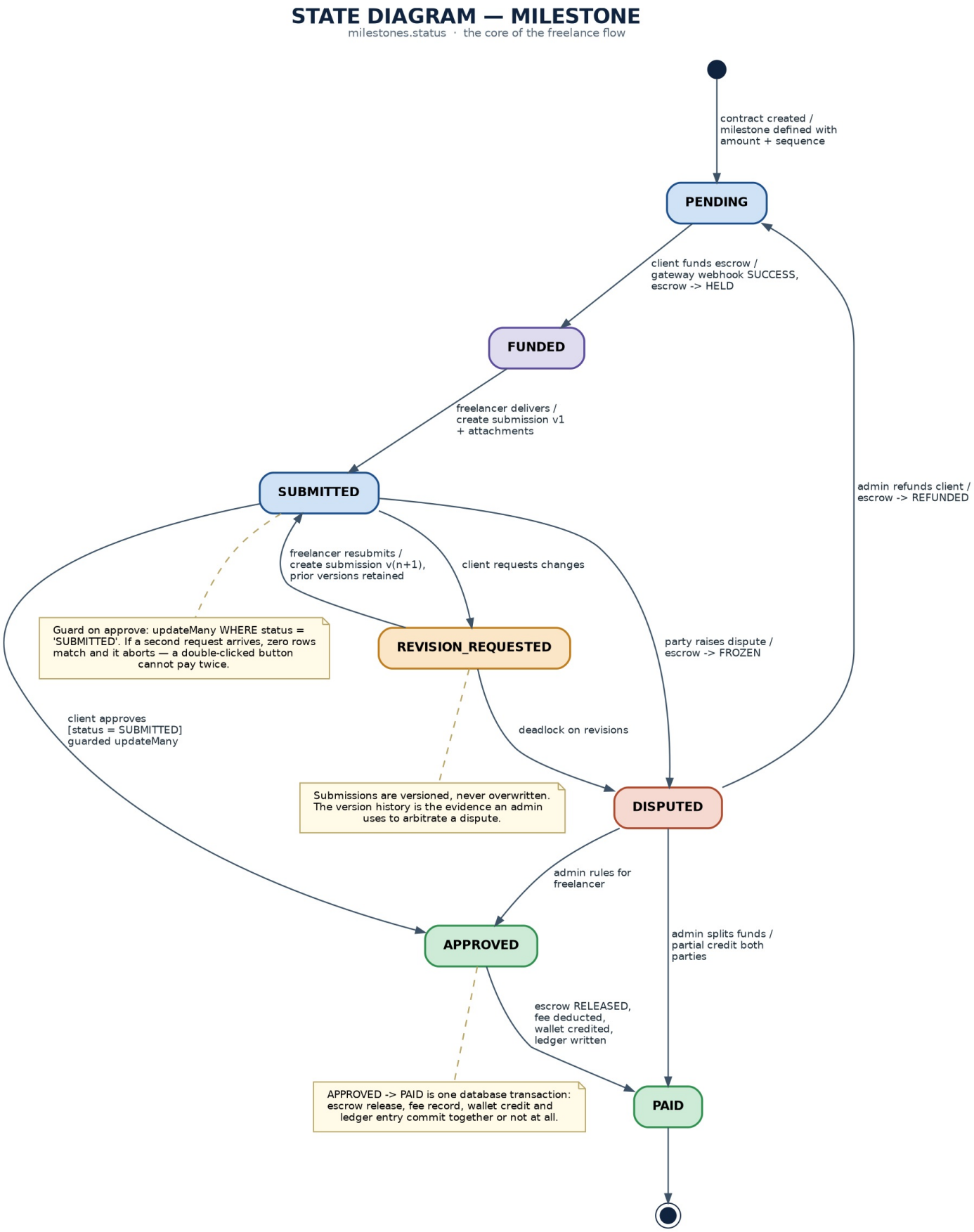


Figure 5 — `milestones.status`

State	Meaning	Permitted next
`PENDING`	Defined but not funded — work should not start	`FUNDED`
`FUNDED`	Escrow holds the money; freelancer may begin	`SUBMITTED`
`SUBMITTED`	Work delivered, awaiting client review	`APPROVED`, `REVISION_REQUESTED`, `DISPUTED`
`REVISION_REQUESTED`	Client wants changes	`SUBMITTED`, `DISPUTED`
`APPROVED`	Accepted; release is in flight	`PAID`
`DISPUTED`	Contested; escrow frozen pending arbitration	`APPROVED`, `PENDING`, `PAID`
`PAID`	Funds released and ledger written	— terminal

6.1 The transitions that must be impossible

This is the more useful half of the diagram, and the part an interviewer will probe.

Invalid transition	Why it must be refused
`PENDING → SUBMITTED`	Work delivered against unfunded escrow. The freelancer has no payment guarantee — the entire reason escrow exists.
`PENDING → PAID`	Payment with no work and no funds held. Money would be created from nothing.
`FUNDED → PAID`	Payment without delivery or approval.
`SUBMITTED → PAID`	Skips client approval. Approval is the client's only control point.
`PAID → anything`	Terminal. Reversing a completed payment must be a new refund record, never a state rollback, or the ledger stops reconciling.
`APPROVED → SUBMITTED`	Un-approving after release has begun would leave escrow and milestone disagreeing.

6.2 The concurrency guard

The `SUBMITTED → APPROVED` transition is where a double-clicked button becomes a double payment. The defence is to make the transition itself conditional, so the database decides:

```
const updated = await tx.milestone.updateMany({
  where: { id: milestoneId, status: 'SUBMITTED' },   // ← the guard
  data:  { status: 'APPROVED', approvedAt: new Date() },
});
if (updated.count === 0) {
  throw new ConflictError('Milestone is not awaiting approval');
}
```

If a second request arrives after the first has committed, the status is already `APPROVED`, zero rows match, and the second request aborts before touching any money. This is **optimistic concurrency control expressed as a state guard**, and it is a strong thing to be able to explain — it shows the state machine is enforced in the database rather than assumed by the UI.

7. Payment, Escrow and Payout

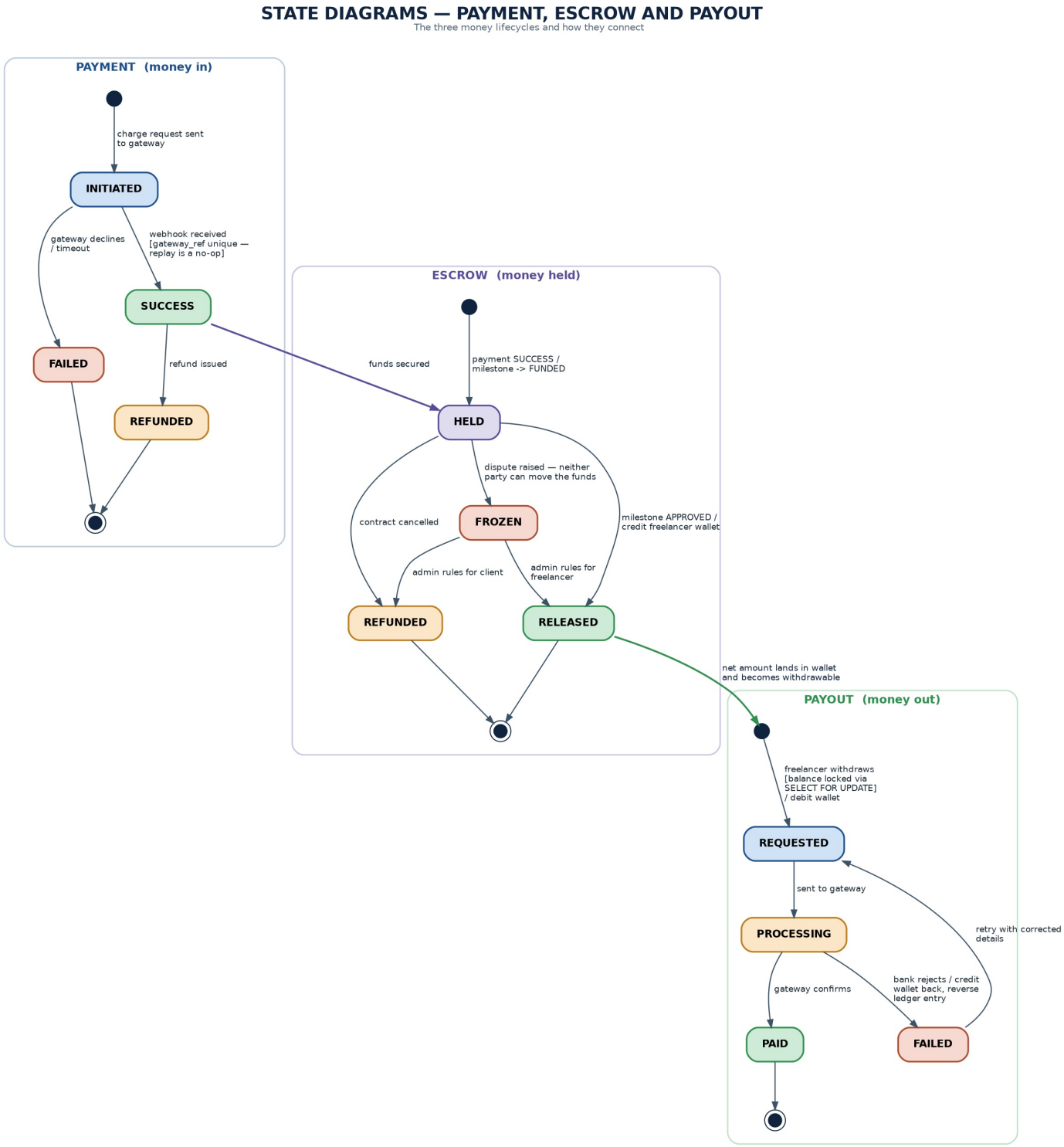


Figure 6 — `payments.status`, `escrows.status`, `payouts.status`

Three separate machines, deliberately. They are chained — a successful payment creates held escrow, a released escrow credits the wallet, a wallet debit starts a payout — but each has its own lifecycle and its own failure modes.

Machine	The state that carries the design weight
Payment	`INITIATED`. A payment is never assumed successful at creation — it is created pending, and only the gateway webhook advances it. The unique `gateway_ref` makes a replayed webhook a no-op.
Escrow	`FROZEN`. The moment a dispute is raised, neither party can move the funds. Without this state, a client could pull funds mid-dispute and the arbitration would be meaningless.
Payout	`FAILED → REQUESTED`. A bank rejection must credit the wallet back and reverse the ledger entry, then allow retry. Forgetting the reversal is how money silently disappears.

The concurrency point, restated because it is the one most often asked. The `REQUESTED` transition on payout reads the wallet balance. A `CHECK (balance >= 0)` does *not* make this safe: two concurrent withdrawals can both read a balance of 500, both pass the check, and both commit, leaving −500 without the constraint ever being violated at any instant either transaction observed. The state transition must acquire a row lock — `SELECT ... FOR UPDATE` — so the second request waits and re-reads. Constraints protect against bad data; locks protect against bad concurrency.

8. Dispute

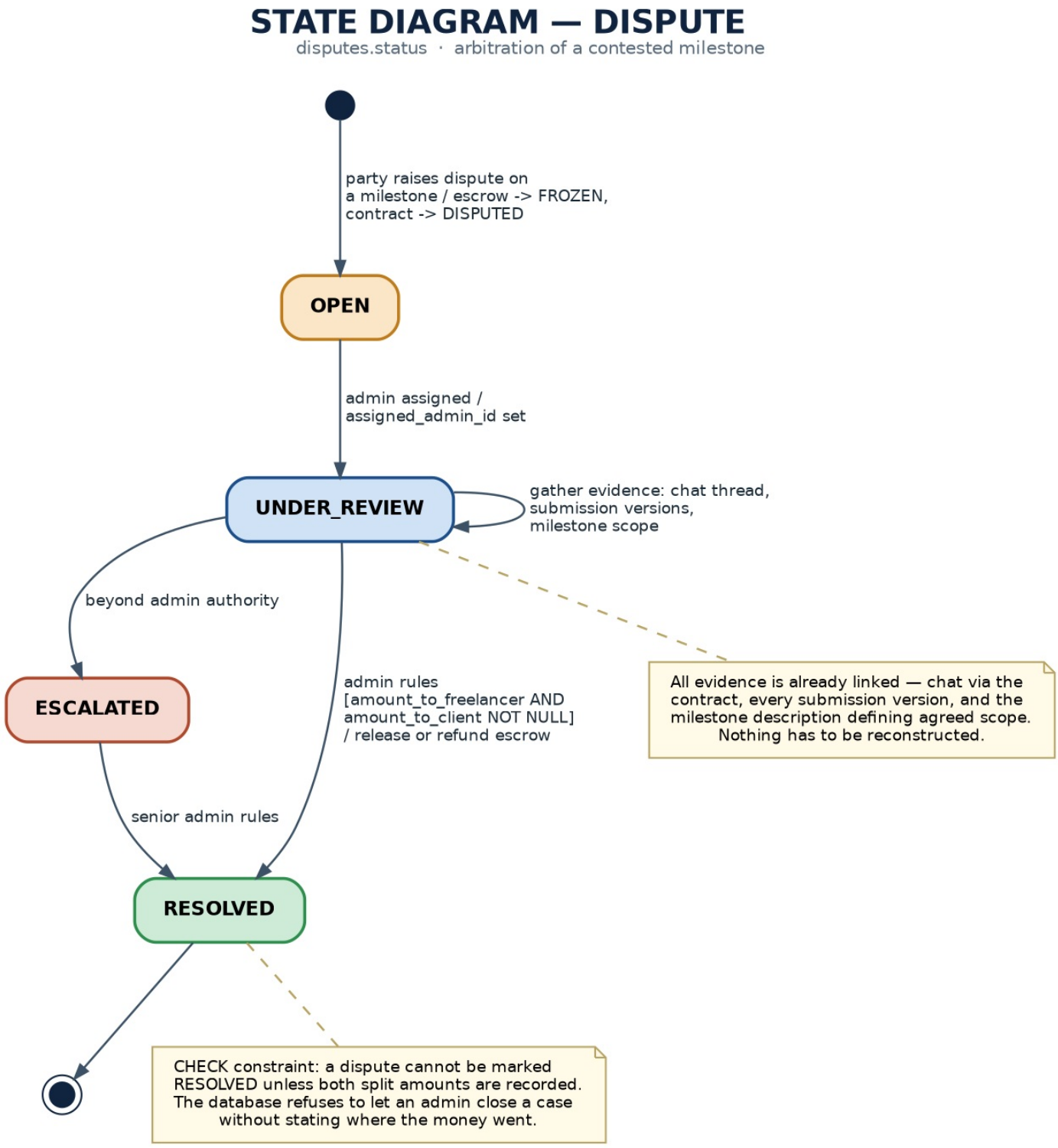


Figure 7 — `disputes.status`

A dispute is the only machine whose transitions are driven mainly by an admin rather than by the transacting parties. Its resolution reaches back into three other machines: the escrow unfreezes and either releases or refunds, the milestone moves to `APPROVED`, `PENDING` or `PAID` depending on the ruling, and the contract returns from `DISPUTED` to `ACTIVE` or moves to `CANCELLED`.

The constraint worth quoting: a dispute cannot enter `RESOLVED` unless both `amount_to_freelancer` and `amount_to_client` are populated. The database refuses to let an admin close a case without recording where the money went — a guard on a state transition expressed as a CHECK constraint rather than as application logic.

9. User Account

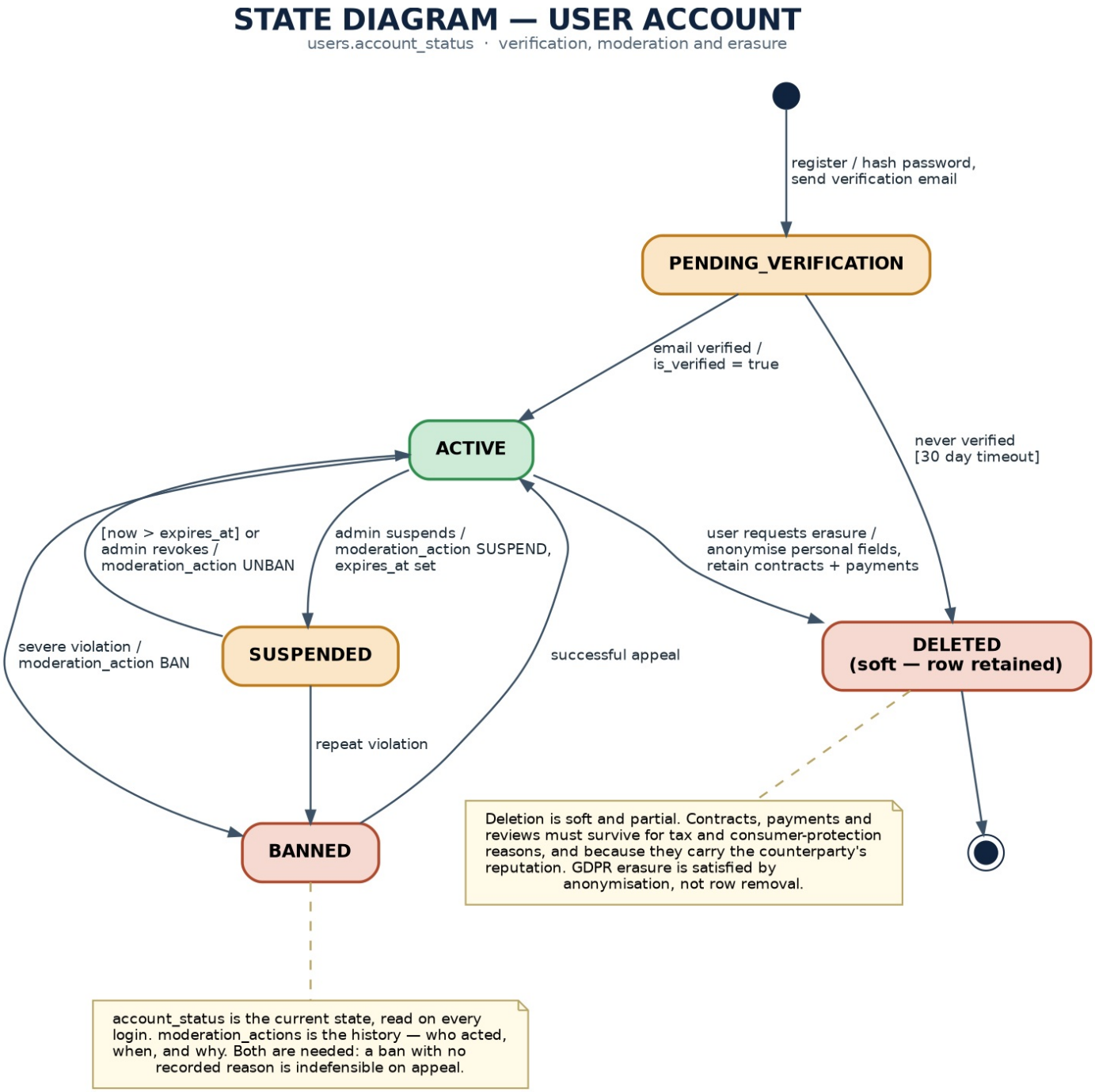


Figure 8 — `users.account_status`

Two aspects are worth drawing attention to. First, `SUSPENDED → ACTIVE` can fire *automatically* once `expires_at` passes — a transition with no human actor, which needs a scheduled job to exist at all. Second, `DELETED` is a soft state: the row remains, personal fields are scrubbed, and contracts, payments and reviews survive.

> **The GDPR nuance, stated properly.** A hard delete would cascade away financial records you are required to retain and would silently rewrite the reputation of every counterparty who worked with that person. The reconciliation between the right to erasure and record-keeping obligations is *anonymisation* — remove the personal data, retain the transactional skeleton. Naming that tension unprompted is a strong signal in an interview.

10. Interview

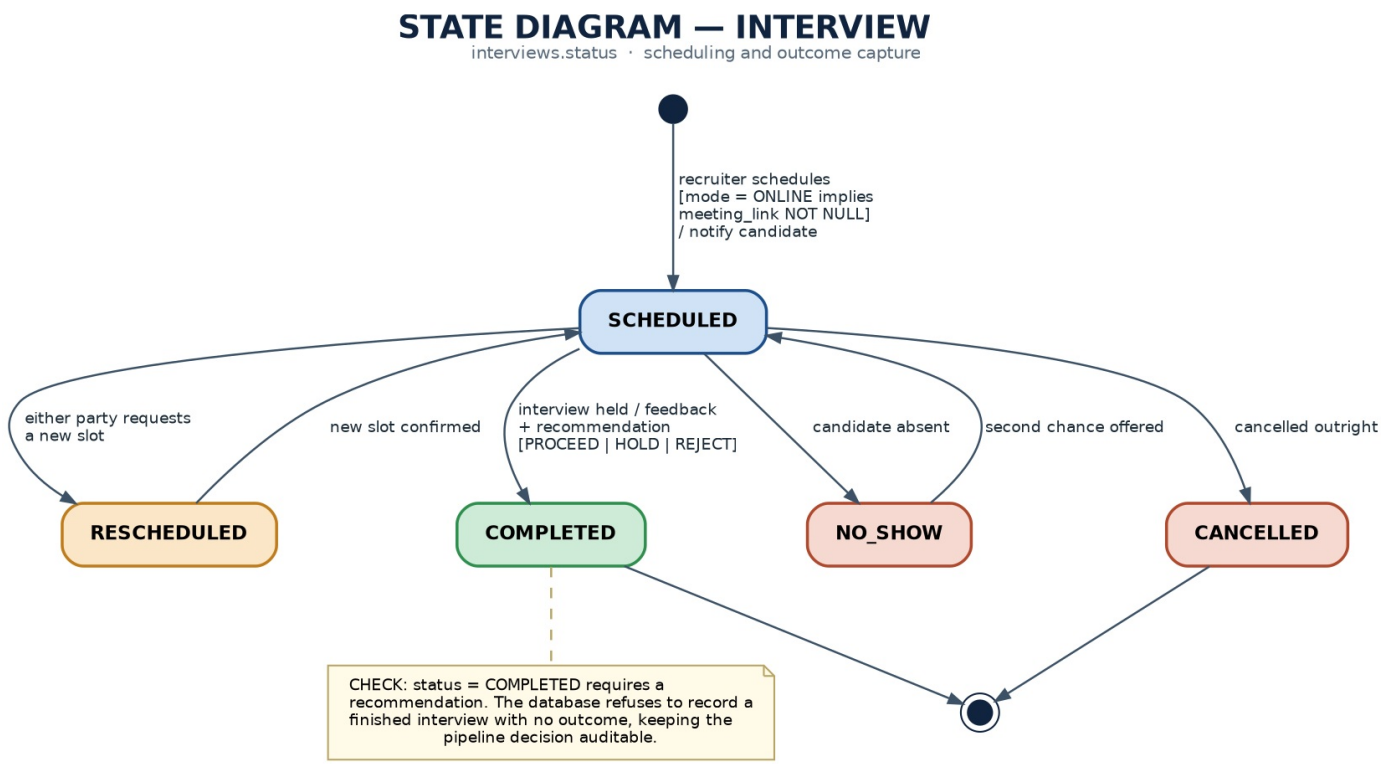


Figure 9 — `interviews.status`

The CHECK constraint requiring a recommendation on `COMPLETED` is the notable part: the database will not record a finished interview with no outcome. That keeps the hiring decision auditable, which matters if a rejection is ever challenged.

11. Composite — Four Machines, One Transaction

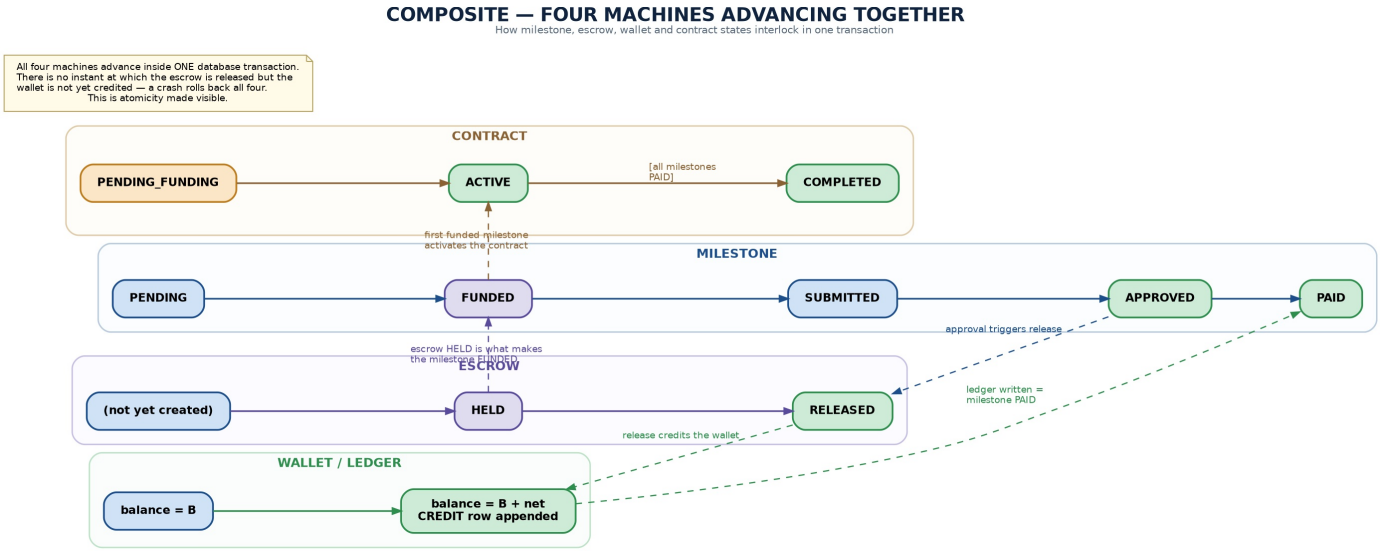


Figure 10 — How milestone, escrow, wallet and contract states advance together

This is the diagram to lead with if asked to explain the hardest part of the system. Four independent state machines advance in lockstep, and the correctness requirement is that they do so **atomically**: there must be no observable instant at which the escrow is `RELEASED` but the wallet has not been credited.

```
await prisma.$transaction(async (tx) => {
  await tx.$queryRaw`SELECT * FROM wallets WHERE user_id = ${uid} FOR UPDATE`;

  // milestone:  SUBMITTED -> APPROVED   (guarded)
  // escrow:     HELD      -> RELEASED
  // wallet:     B         -> B + net
  // ledger:     append CREDIT row with balance_after
  // contract:   ACTIVE    -> COMPLETED  [if all milestones PAID]
});
```

A crash at any point rolls back all four. That is atomicity — the "A" in ACID — demonstrated on your own system rather than recited from a definition.

12. Implementing State Machines Correctly

12.1 Define transitions as data, not scattered conditionals

```
const MILESTONE_TRANSITIONS: Record<MilestoneStatus, MilestoneStatus[]> = {
  PENDING:            ['FUNDED'],
  FUNDED:             ['SUBMITTED'],
  SUBMITTED:          ['APPROVED', 'REVISION_REQUESTED', 'DISPUTED'],
  REVISION_REQUESTED: ['SUBMITTED', 'DISPUTED'],
  APPROVED:           ['PAID'],
  DISPUTED:           ['APPROVED', 'PENDING', 'PAID'],
  PAID:               [],
};

function assertTransition(from: MilestoneStatus, to: MilestoneStatus) {
  if (!MILESTONE_TRANSITIONS[from].includes(to)) {
    throw new ConflictError(`Cannot move milestone from ${from} to ${to}`);
  }
}
```

One table, one check, one place to change. Scattering `if (status === 'SUBMITTED')` across a dozen handlers guarantees that some path eventually forgets a case.

12.2 Three layers of enforcement

Layer	Enforces	Why it is not sufficient alone
UI	Hides invalid actions	Trivially bypassed; it is a convenience, not a control
Service	The transition table above	Does not survive concurrency — two requests can both pass the check
Database	Guarded `updateMany`, row locks, CHECK constraints	Poor error messages, but it is the only layer that holds on every path

12.3 Always record the transition, not just the new state

For anything auditable, write a history row alongside the status update, in the same transaction — `stage_movements` for applications, `transactions` for wallets, `milestone_submissions` for deliveries, `moderation_actions` for bans. The status column tells you *where* something is; the history tells you *how it got there*, which is what you need when a decision is challenged.

13. Interview Questions

1. What does a state diagram show that your ER and DFD do not?

Legal transitions over time. The ER diagram says a milestone has a status column; the DFD says approval data flows into the payment process. Neither says that `PENDING → PAID` must be impossible. The state diagram is the specification of what my service layer has to refuse.

2. Walk me through the milestone lifecycle.

`PENDING` when the contract is created, `FUNDED` once the client's escrow payment clears, `SUBMITTED` when the freelancer delivers, then either `REVISION_REQUESTED` and back to `SUBMITTED`, or `APPROVED` on acceptance, then `PAID` once escrow is released and the ledger written. `DISPUTED` can branch off from `SUBMITTED` or `REVISION_REQUESTED` and resolves back into approval, refund or a split.

3. Which transitions did you deliberately make impossible, and why?

`PENDING → SUBMITTED`, because that is work delivered against unfunded escrow and the freelancer has no guarantee of payment. `SUBMITTED → PAID`, because it skips the client's only control point. And anything out of `PAID`, because reversing a completed payment must be a new refund record rather than a state rollback — otherwise the ledger stops reconciling.

4. Two approve requests arrive at once. What happens?

The transition is a guarded update: `updateMany WHERE id = ? AND status = 'SUBMITTED'`. The first commits and flips the status. The second matches zero rows, so the count is zero and the service throws a conflict before touching any money. The state guard is doing optimistic concurrency control, enforced by the database rather than assumed by the UI.

5. Why is `FROZEN` a separate escrow state?

Because during a dispute neither party may move the funds. Without it, the client could pull the money back mid-arbitration and the admin's ruling would be unenforceable. It is a state that exists purely to remove capability from both parties, which is a useful thing to be able to recognise.

6. Your application diagram has a self-loop on IN_PROGRESS. Isn't that hiding something?

Yes — stage movement is a sub-machine, but its states are data rather than enum values, because each recruiter configures their own pipeline. You cannot draw a fixed diagram for states defined at runtime, which is exactly why `hiring_stages` and `stage_movements` are tables rather than more enum values.

7. Where do you enforce these transitions?

Three layers. The UI hides invalid actions, which is convenience only. The service checks a transition table, which gives good error messages but does not survive concurrency. The database enforces it through guarded updates, row locks and CHECK constraints, which is the only layer that holds on every path — including migrations, admin scripts and background jobs that never touch my service code.

8. Is `DISPUTED` a terminal state on the contract?

No, and that is deliberate. A contract returns to `ACTIVE` once the admin resolves the dispute, because the remaining milestones still need doing. Only if the admin terminates the engagement does it go to `CANCELLED`. Modelling a dispute as a detour rather than an ending matches how disputes actually resolve.

9. Which transitions have no human actor?

Invitation and offer expiry, and the automatic lift of a temporary suspension when `expires_at` passes. These need a scheduled job to exist at all, and they are easy to forget precisely because no user triggers them. Their absence shows up as invitations that stay open forever and suspensions that never lift.

10. How do you keep the four money machines consistent?

They advance in a single database transaction — milestone, escrow, wallet and ledger, plus the contract if it is the final milestone. There is no observable instant at which escrow is released but the wallet is not credited, because a crash rolls back all four. That is atomicity demonstrated rather than recited.

> **Preparation advice.** Learn two diagrams properly rather than all ten superficially: the **milestone** machine, because it is where correctness actually matters, and the **composite**, because it shows four machines advancing atomically. The strongest single answer you can give is question 3 — naming the transitions you made *impossible*, and what would go wrong if they were allowed. Anyone can list states; knowing which moves must be refused is what shows you designed the machine rather than transcribed an enum.